



DAYANANDA SAGAR COLLEGE OF ARTS SCIENCE AND COMMERCE

Shavige Malleshwara Hills, 1st Stage, Kumaraswamy Layout, Bengaluru,
Karnataka.

Department of Computer Application-MCA(BU)

**Project Report Title : Simulation of a Simple Network Protocol using Finite
State Machine(FSM)**

Submitted To,

Ms.Akshatha,
Assistant Professor,
DSCASC.

Submitted by,

Rasika Mala K(P03CJ24S126079)
Usha.S.P(P03CJ24S126111)
Vyshnavi.G(P03CJ24S126116)

In-charge Sign

Table of content

S. No.	Section Title	Description
1	Problem Statement	-
2	Abstract	Glimpse of the project
3	Introduction	Relevance of FSM in networks, project scope
4	Objective	Aim and key goals of the project
5	Application of TOC in computer Networks	How FSM and TOC principles apply to network protocol design
6	System Design	Architecture and FSM breakdown
7	Implementation	Tools, programming approach and code(python)
8	Output	output analysis
9	Conclusion	Summary of learnings and outcomes

Problem statement

Discuss the application of TOC on computer network in detail Design and Implement a **Finite State Machine (FSM)** for a Simple Network Protocol Simulation.

Objective: The goal of this project is to design and simulate a simple network communication protocol using a **finite state machine (FSM)** approach. The system should model a basic communication process between two devices over a network, including states like Idle, Connecting, Transmitting Data, and Disconnected. Project Description:

You are required to implement a **finite state machine (FSM)** that simulates a network protocol. The simulation should cover basic operations such as establishing a connection, data transfer, and termination. The project should also handle state transitions based on specific events (e.g., sending a "Connect" message, "Data" transmission, or "Disconnect").

Abstract

This project showcases a simulation of a basic **network communication protocol** through a **Finite State Machine (FSM)** model. In computer networks, communication protocols specify the protocol for data exchange among devices. Such protocols tend to be state-based in nature, hence FSM is the best modeling tool for them. The system in this project simulates a communication process based on four core states: **Idle, Connecting, Transmitting, and Disconnected**. Every state transition is caused by a particular network event like connection request, acknowledgment, data transmission, or timeout. **The FSM is coded in Python** and verified against several sequences of events to check its behavior. Results show how the FSM can represent a safe, predictable communication process. This simulation not only emphasizes the real-world application of **Theory of Computation (TOC)** in networking but also lays the groundwork for further investigation into actual protocol design and verification.

Introduction

Computer networks in our interconnected world today are essential to facilitate communication between devices. Such communication is controlled by **network protocols**—a **collection of predetermined rules and procedures that guarantee data exchange reliability**. In many of these protocols, there is a **Theory of Computation (TOC)** concept called the **Finite State Machine (FSM)**.

A Finite State Machine is a computational model made up of a finite set of states, transitions among those states, and actions. It is used extensively across several areas of computer science, like parsing languages, control systems, and particularly network protocol specification. FSMs offer a formal and organized way to model how systems behave as input or events from the outside world interact with them, making them perfect for describing the behavior of communication protocols.

In the context of computer networks, FSMs assist in representing the stateful behavior between devices. For instance, a device can begin in an **"Idle"** state, transition to a **"Connecting"** state when establishing a connection, change to **"Transmitting"** after sending data, and end in a **"Disconnected"** state when closing the session. Each transition takes place upon the occurrence of particular events such as receiving acknowledgments, sending data, or timing out.

This project investigates the application of FSMs to simulate a simple network communication protocol. Through the design and implementation of a finite state machine, we seek to abstract the common lifecycle of a network session in a simplified and pedagogical way. The simulation embodies the spirit of protocol design and illustrates how formal models such as FSMs can introduce order and clarity into system behavior.

Objective

The goal of this project is to emulate **a simple network communication protocol using FSM**.
The FSM ought to:

- Emulate simple states: Idle, Connecting, Transmitting, and Disconnected.
- Deal with state changes initiated by network events.
- Emulate client-server interaction in real life but in a simplified manner.

Application of TOC in Computer Networks

TOC facilitates the formal definition of formal models such as FSMs, which serve as a basis for:

- **Protocol Design:** Protocols such as TCP/IP, HTTP, and FTP are based on rules of FSM-like state transitions.
- **Error Detection/Recovery:** FSMs support timeouts, retries, and error handling for invalid states.
- **Efficient Implementation:** FSM-based protocols may be implemented with low-overhead code and hardware logic.
- **Verification:** FSMs facilitate the ease of verification of protocol correctness with formal techniques.

System Design

To model a basic network protocol with a Finite State Machine (FSM), the system is structured around a couple of basic elements: states, events, and transitions. States are each a stage in the network communication process, and events cause transitions between states.

FSM Components

❖ States

Each state within the FSM is a specific condition or mode of operation within the protocol:

1. Idle

- The starting state.
- Represents a device in an idle state waiting to send or receive a connection.
- No communication is occurring in this state.

2. Connecting

- Represents the state in which a device is trying to connect.
- A "connect" message is usually sent, and the device waits for acknowledgment.
- It may move to either Transmitting (if successful) or Disconnected (if it fails).

3. Transmitting

- This is the state of active communication.
- Data packets are sent and perhaps received in this state.
- It stays in this condition until a "disconnect" event occurs.

4. Disconnected

- Indicates the termination of communication.
- The connection has been closed or has failed.
- The FSM can transition back to the Connecting or Idle state from here depending on subsequent events.

❖ Events

Events are the inputs to the FSM that make it change states. These simulate real-world protocol operations:

- **connect_request**: Sent when a device is requesting to open a connection.
- **ack_received**: Receipt of acknowledgment from the other device, that the connection has been established.
- **send_data**: Data transmission indicated.
- **disconnect**: Request for ending the connection.
- **timeout**: Failure or lag in receiving a response, leading to ending the ongoing session.

❖ State Transition Logic

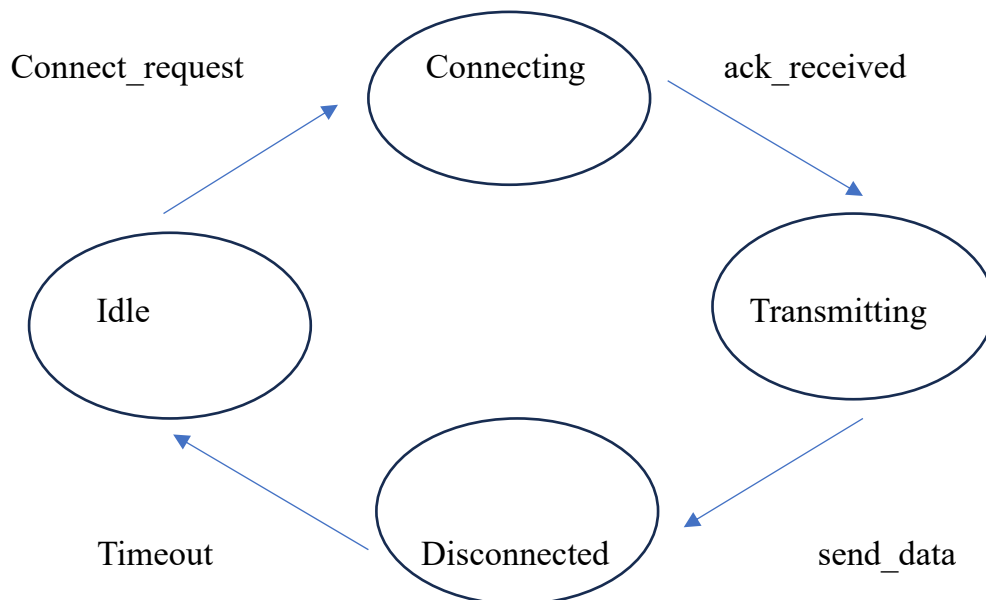
Transitions define how the FSM moves from one state to another based on events:

Current State	Event	Next State	Action/Description
Idle	connect_request	Connecting	Start connection attempt
Connecting	ack_received	Transmitting	Connection establishment
Connecting	timeout	Disconnected	Connection failed or timed out
Transmitting	send_data	Transmitting	Data is sent, remains in Transmitting state
Transmitting	disconnect	Disconnected	Connection is closed
Disconnected	connect_request	Connecting	Reattempt to connect
Disconnected	timeout	Idle	Reset back to idle due to prolonged failure

❖ Design Considerations

- **Simplicity:** The design specifically employs only four fundamental states to keep the simulation simple to comprehend and use.
- **Reusability:** The FSM is designed to be modular, so it can be easily extended in the future (e.g., adding an "Error" state or "Waiting for Acknowledgment").
- **Scalability:** Although simple, the model captures the underlying principles behind actual protocols such as TCP/IP or HTTP, which can be built upon this structure.

❖ FSM Diagram



Implementation(python code example)

```
class NetworkProtocolFSM:
    def __init__(self):
        self.state = 'Idle'

    def on_event(self, event):
        print(f'Current State: {self.state} | Event: {event}')

        if self.state == 'Idle':
            if event == 'connect_request':
                self.state = 'Connecting'

        elif self.state == 'Connecting':
            if event == 'ack_received':
                self.state = 'Transmitting'
            elif event == 'timeout':
                self.state = 'Disconnected'

        elif self.state == 'Transmitting':
            if event == 'send_data':
                print("Data sent.")
            elif event == 'disconnect':
                self.state = 'Disconnected'

        elif self.state == 'Disconnected':
            if event == 'connect_request':
                self.state = 'Connecting'

        print(f'New State: {self.state}\n')

# --- Simulation ---
fsm = NetworkProtocolFSM()
```

```
fsm.on_event('connect_request') # Should transition to 'Connecting'
fsm.on_event('ack_received')    # Should transition to 'Transmitting'
fsm.on_event('send_data')      # Should print "Data sent."
fsm.on_event('disconnect')     # Should transition to 'Disconnected'
fsm.on_event('connect_request') # Should transition back to 'Connecting'
fsm.on_event('timeout')        # Should transition to 'Disconnected'
```

❖ Code Output

```
Current State: Idle | Event: connect_request
New State: Connecting

Current State: Connecting | Event: ack_received
New State: Transmitting

Current State: Transmitting | Event: send_data
Data sent.
New State: Transmitting

Current State: Transmitting | Event: disconnect
New State: Disconnected

Current State: Disconnected | Event: connect_request
New State: Connecting

Current State: Connecting | Event: timeout
New State: Disconnected
```

Conclusion

This project illustrates the applicability of Theory of Computation (TOC) principles—namely Finite State Machines (FSMs)—to model and simulate simple network communication protocols. By encapsulating the protocol into a finite number of states and well-defined transitions, we were able to develop a system that emulates the actual-world behavior of devices communicating on a network.

With the simulation, we effectively implemented four significant states—Idle, Connecting, Transmitting, and Disconnected—and established event-driven transitions governing how the system acts as a response to standard networking activities such as connecting, data sending, disconnecting, and timeout handling. The FSM model presented a formal, deterministic mechanism to define these interactions in such a way that each potential input maps to a defined system state.

The testing phase ensured that the FSM implementation responds to valid transitions correctly, remains stable in working states, and properly resets on failure. This ensures the appropriateness of FSMs not merely for theoretical computation problems but also for real-world, real-time applications like protocol design, system control, and event-driven software.

Aside from this particular realization, the project also supports the general value of TOC ideas in contemporary computer science. FSMs are the basis for more sophisticated systems, such as parsers, real-time operating systems, communication stacks (e.g., TCP/IP), and even AI state planners. This project is a microcosm of those larger systems, providing insight into how abstract models can inform and improve practical software engineering.

Key Takeaways:

- FSMs are a useful abstraction for describing stateful systems like network protocols.
- Formal models enhance correctness and clarity, making it simpler to test and reason about systems.
- Protocols in the real world are often based on the same state/event mechanisms modeled here.
- This project gives a good grounding for learning more advanced subjects like protocol verification, distributed systems, and concurrent computing.

In summary, the application of TOC concepts such as FSMs to network protocol design not only facilitates a better understanding of how networks work but also enables students and developers to utilize tools for creating dependable, predictable, and maintainable systems.